Terminal Reader API — Specification

Calypso Networks Association

Version 3.0.0-SNAPSHOT, 2026-06-04

Table of Contents

Chapter 1. Abstract

This document is the official technical specification of the **Terminal Reader API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way, the interfaces, classes, enumerations, exceptions, structural relationships and behavioural requirements that any conforming implementation of the Terminal Reader API MUST satisfy.

The Terminal Reader API is part of the broader CNA **Terminal API** family and acts as the contract through which terminal applications discover card readers, observe card events, select cards and exchange data with them.

Document Status

Specification ID	CNA-TR-API
Version	3.0.0-SNAPSHOT
Status	Working Draft
Editor	Calypso Networks Association
Reference model	`api_class_diagram.puml` / `api_class_diagram.svg`
Reference license	MIT License

Revision History

Version	Date	Editor	Description
1.0.0	*TBC*	CNA	Initial release of the Terminal Reader API.
2.0.0	*TBC*	CNA	Introduction of the `ReaderApiFactory` and of the `CardSelector` hierarchy.
2.1.0	*TBC*	CNA	Introduction of `ChannelControl`, `InvalidCardResponseException` and `CardTransactionManager`.
3.0.0-SNAPSHOT	*TBD*	CNA	*Work in progress — see Appendix A.*

Chapter 2. Introduction

2.1. Purpose

The Terminal Reader API defines a standardised set of contracts allowing terminal applications to interact with card readers and the cards presented to them, independently of:

- the underlying reader hardware (contact, contactless, virtual, etc.),
- the card technology family (ISO/IEC 7816-4, ISO/IEC 14443, proprietary),
- the implementation technology of the application (programming language, runtime).

Conforming implementations of this specification MUST allow extension modules (e.g. card-specific APIs such as the Calypso Card API) to plug in transparently through the Service Provider Interface (SPI) mechanism defined in Section 5.4.

2.2. Scope

This specification covers:

- the discovery and basic interrogation of card readers,
- the observation of reader events related to card presence and selection,
- the configuration of card detection (where applicable),
- the description and orchestration of card selection scenarios,
- the abstraction of the physical and logical communication channels established with cards,
- the contract surface required by card extension modules to interoperate with reader implementations.

The following topics are **out of scope**:

- the wire-level protocols used between reader and card (e.g. APDU framing, ISO/IEC 14443 anti-collision),
- the cryptographic operations performed by or on behalf of the card,
- the lifecycle and packaging of reader drivers,
- the user interface presented by the terminal application.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises exceptions exclusively of the types defined in Chapter 6 for the situations described,
3. it preserves the structural and behavioural relationships described in Chapter 4.

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in **RFC 2119** and **RFC 8174** when, and only when, they appear in all capitals.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced exactly as they appear in the reference UML class diagram (Section 4.3).

2.4.2. Language-agnostic types

The following symbolic type names are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Symbolic type	Description	Typical bindings
`string`	Sequence of characters, UTF-8 encoded by default.	`String` (Java), `string` (C#), `str` (Rust/Python).
`boolean`	Two-state truth value.	`boolean`, `bool`.
`integer`	Signed 32-bit integer.	`int`, `Int32`.
`byte array`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`set<T>`	Unordered collection of unique `T` values.	`Set<T>`, `HashSet<T>`.
`list<T>`	Ordered collection of `T` values, may contain duplicates.	`List<T>`, `Vec<T>`.
`map<K, V>`	Associative array binding keys of type `K` to values of type `V`.	`Map<K,V>`, `Dictionary<K,V>`.
`void`	Indicates that an operation returns no value.	`void`, `Unit`, `None`.

2.4.3. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation’s language-agnostic signature.
Since	The version of the specification that introduced the operation.
Description	A normative description of the operation’s behaviour.
Parameters	A description of each input parameter.

Returns	A description of the returned value, if any.
Throws	A list of exceptional conditions, each mapped to a specific exception type.

2.4.4. Element status markers

Marker	Meaning
(no marker)	Stable element belonging to the specification baseline.
`[new]`	Element added in 3.0.0-SNAPSHOT.
`[deprecated]`	Element retained for backward compatibility but scheduled for removal.
`[draft]`	Element under active discussion; signature or semantics MAY still change.

2.5. References

Reference	Identifier	Title
[RFC 2119]	RFC 2119	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	RFC 8174	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 7816-4]	ISO/IEC 7816-4	*Identification cards — Integrated circuit cards — Part 4: Organization, security and commands for interchange*.
[ISO/IEC 14443]	ISO/IEC 14443	*Identification cards — Contactless integrated circuit cards — Proximity cards*.
[CNA-CARD]	CNA-CARD-API	*Calypsonet Terminal Card API Specification*, Calypso Networks Association.
[CNA-DEF]	CNA-DEF-API	*Calypsonet Terminal Definitions API Specification*, Calypso Networks Association.
[KEYPOP]	KEYPOP	Keypop open-source project, hosted by the Eclipse Foundation — https://keypop.org.

Chapter 3. Terms, Definitions and Abbreviations

3.1. Terms and Definitions

Term	Definition
Card	Any physical or virtual smart card that can be presented to a reader for selection and dialogue.
Card Reader	Hardware or software component capable of detecting cards and exchanging data with them.
Card Selection	Sequence of operations used to identify, filter and prepare a card for application-level dialogue.
Logical Channel	Software-level communication path established with a card after a successful selection.
Physical Channel	Hardware-level communication path between the reader and the card.
Observable Reader	Reader implementation able to notify observers asynchronously of card-related events.
Power-on Data	Data returned by the reader at the moment a card is detected (ATR, virtual ATR, anti-collision data).
Card Application	Application Dedicated File (DF) selected on a card, identified by its AID, as defined in ISO/IEC 7816-4.
AID	Application Identifier as defined in ISO/IEC 7816-4 (5 to 16 bytes).
APDU	Application Protocol Data Unit, the message format used between the terminal and the card.
SPI	Service Provider Interface — a set of interfaces designed to be implemented by an extension module.

3.2. Abbreviations

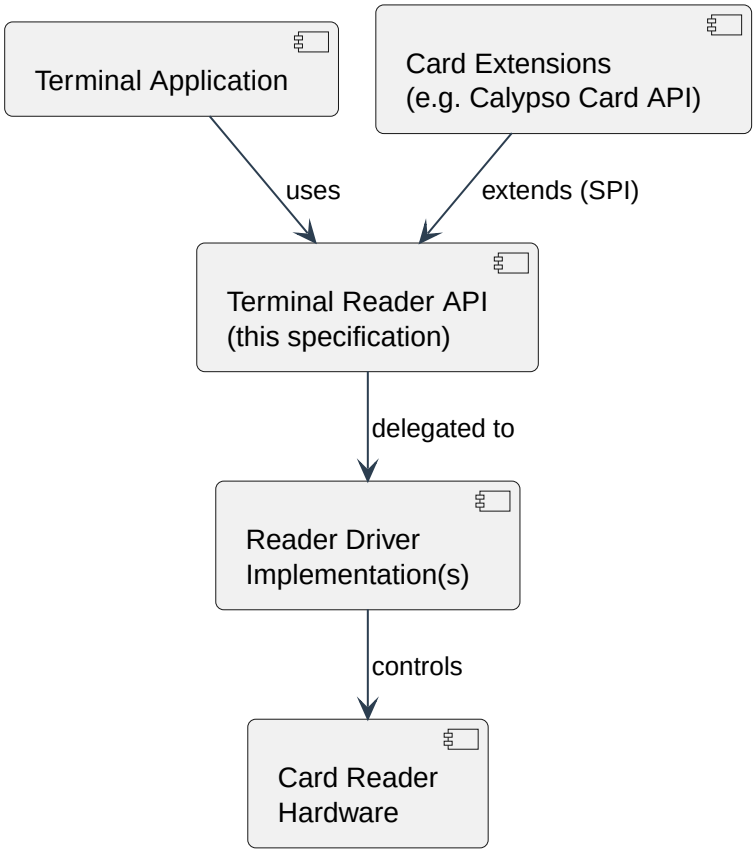
Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
ATR	Answer To Reset
CNA	Calypso Networks Association
DF	Dedicated File
FCI	File Control Information
FCP	File Control Parameters
FMD	File Management Data

Abbreviation	Expansion
ISO	International Organization for Standardization
RFC	Request For Comments
SPI	Service Provider Interface
UML	Unified Modelling Language

Chapter 4. Architectural Overview

4.1. Functional positioning

The Terminal Reader API sits between the terminal application and the reader drivers, exposing a stable contract on each side:



4.2. Logical packages

The specification is organised in the following logical packages. Each package groups elements that share a common responsibility and a common stability lifecycle.

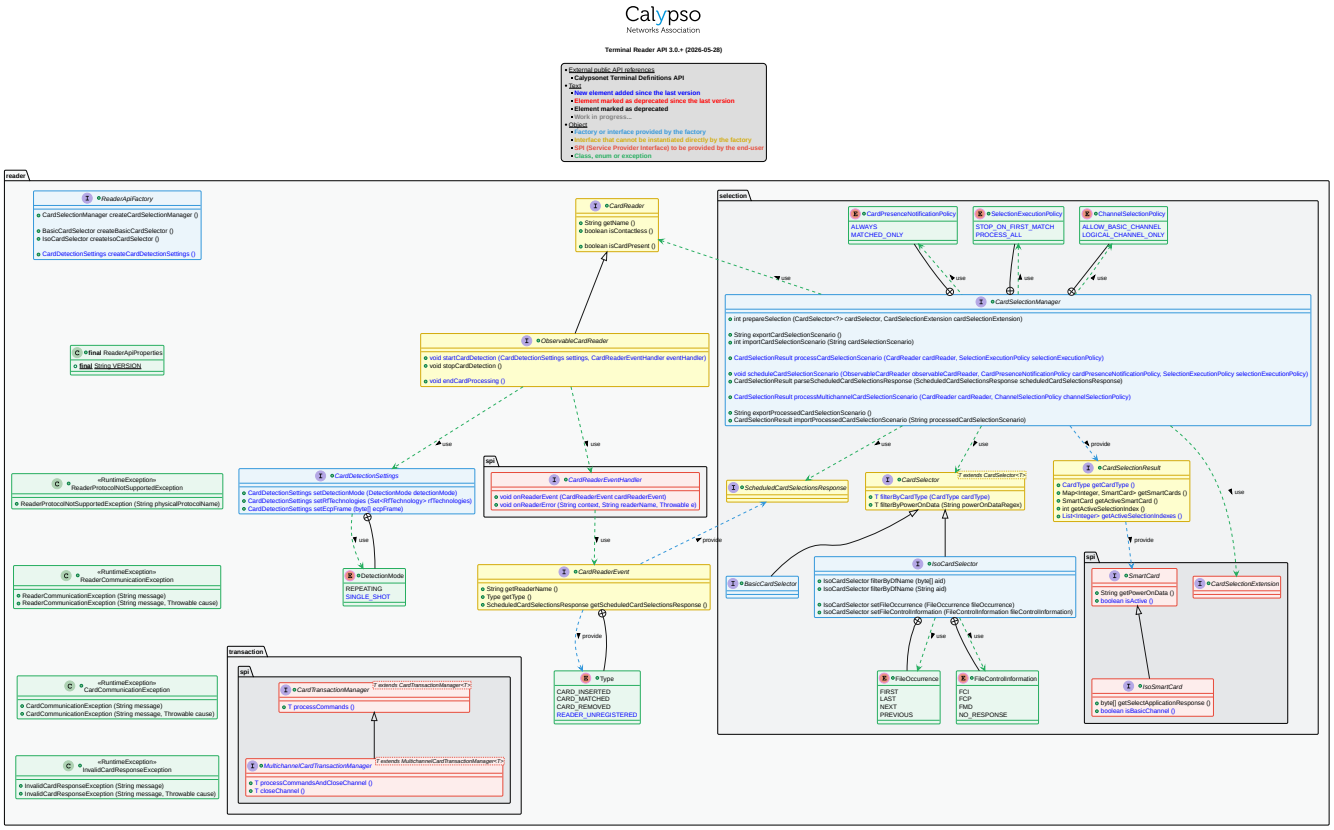
Package	Role	Summary
`reader`	Public API	Reader discovery, identification, configuration and observation.
`reader.spi`	SPI	Interfaces implemented by application code to consume reader events.
`reader.selection`	Public API	Definition, orchestration and parsing of card selection scenarios.
`reader.selection.spi`	SPI	Interfaces implemented by card extension modules to enrich the selection process.

Package	Role	Summary
`reader.transaction.spi`	SPI	Common contract shared by all card transaction managers exposed by card extensions.

4.3. Reference class diagram

The normative class diagram of this specification is provided as a PlantUML source file alongside this document:

- `api_class_diagram.puml` — canonical UML model,
- `api_class_diagram.svg` — rendered representation,
- `api_class_diagram_diff.puml` — delta against the previous version.



Chapter 5. API Specification

5.1. Package reader

The `reader` package defines the entry points of the API and the abstractions used to manipulate readers and the events they produce.

5.1.1. ReaderApiProperties

Kind	Final class
Package	`reader`
Since	1.0.0
Purpose	Exposes the immutable properties of the Terminal Reader API.

5.1.1.1. Constants

Name	Type	Since	Description
`VERSION`	`string`	1.0.0	String representation of the version of the API implemented by the conforming binding (e.g. `"3.0"`). The value MUST be a dotted decimal of the form `MAJOR.MINOR`.

The `ReaderApiProperties` type SHALL NOT be instantiated. Implementations MUST prevent external construction.

5.1.2. ReaderApiFactory

Kind	Interface
Package	`reader`
Since	2.0.0
Purpose	Factory used by the application to obtain instances of the public types provided by the API.

The `ReaderApiFactory` is the **only** normative entry point through which the application is allowed to instantiate the public types of the API. Implementations MUST expose exactly one instance of this factory and MUST guarantee that every invocation returns a fresh, fully initialised object.

5.1.2.1. Operations

Signature	`createCardSelectionManager() → CardSelectionManager`
Since	2.0.0
Description	Returns a new, freshly initialised `CardSelectionManager`.

Returns	A non-`null` `CardSelectionManager`.
Throws	None.

Signature	`createBasicCardSelector() → BasicCardSelector`
Since	2.0.0
Description	Returns a new, freshly initialised `BasicCardSelector`.
Returns	A non-`null` `BasicCardSelector`.
Throws	None.

Signature	`createIsoCardSelector() → IsoCardSelector`
Since	2.0.0
Description	Returns a new, freshly initialised `IsoCardSelector`.
Returns	A non-`null` `IsoCardSelector`.
Throws	None.

5.1.3. CardReader

Kind	Interface
Package	`reader`
Since	1.0.0
Purpose	Represents a card reader capable of driving the underlying hardware to manage card detection.

5.1.3.1. Operations

Signature	`getName() → string`
Since	1.0.0
Description	Returns the name of the reader.
Returns	A non-empty `string` identifying the reader. The value MUST be stable for the lifetime of the reader instance.
Throws	None.

Signature	`isContactless() → boolean`
Since	1.0.0
Description	Indicates whether the card communication mode is contactless.
Returns	`true` if the communication mode is contactless, `false` otherwise.
Throws	None.

Signature	`isCardPresent() → boolean`
Since	1.0.0
Description	Indicates whether a card is currently present in the reader.
Returns	`true` if a card is inserted in the reader, `false` otherwise.
Throws	`ReaderCommunicationException` — if the communication with the reader has failed.

5.1.4. ConfigurableCardReader

Kind	Interface
Package	`reader`
Since	1.0.0
Extends	`CardReader`
Purpose	A `CardReader` exposing operations to manage the card communication protocols.

The association between **physical protocol name** (as known by the reader) and **logical protocol name** (as defined by the application) is intended to allow applications to refer to non-ISO cards by a stable, implementation-agnostic identifier when constructing a `CardSelector`.

5.1.4.1. Operations

Signature	`activateProtocol(physicalProtocolName: string, logicalProtocolName: string) → void`
Since	1.0.0
Description	Activates the reader communication protocol by associating the provided physical communication protocol name with the logical communication protocol name defined by the application. The operation: - activates the detection of cards using the provided physical communication protocol; - internally associates the two strings provided as arguments.
Parameters	`physicalProtocolName` — name of the physical communication protocol as known by the reader. Refer to the reader documentation for the list of supported communication protocols. `logicalProtocolName` — name of the logical protocol associated with the cards detected using the physical protocol. This name MAY be used by the application to perform filtering at the time of selection.

Throws	`IllegalArgumentException` — if either provided protocol name is `null` or empty. `ReaderProtocolNotSupportedException` — if the physical communication protocol is not supported.

Signature	`deactivateProtocol(physicalProtocolName: string) → void`
Since	1.0.0
Description	Deactivates the reader communication protocol. The reader MUST stop detecting cards using this protocol.
Parameters	`physicalProtocolName` — name of the physical communication protocol as known by the reader.
Throws	`IllegalArgumentException` — if the provided protocol name is `null` or empty. `ReaderProtocolNotSupportedException` — if the physical communication protocol is not supported.

Signature	`getCurrentProtocol() → string`
Since	1.2.0
Description	Returns the name of the physical protocol currently used by the reader.
Returns	The active physical protocol name, or `null` if no selection has been made yet or no protocol has been activated.
Throws	None.

5.1.5. ObservableCardReader

Kind	Interface
Package	`reader`
Since	1.0.0
Extends	`CardReader`
Purpose	A `CardReader` capable of observing the insertion and removal of cards.

5.1.5.1. Operations

Signature	`setReaderObservationExceptionHandler(exceptionHandler: CardReaderObservationExceptionHandlerSpi) → void`
Since	1.0.0
Description	Registers the exception handler that will receive notifications of fatal errors raised during the observation. Calling this operation is **mandatory** before the reader can be observed.
Parameters	`exceptionHandler` — the handler implemented by the application.
Throws	`IllegalArgumentException` — if the provided handler is `null`.

Signature	`addObserver(observer: CardReaderObserverSpi) → void`
Since	1.0.0
Description	Registers an observer to be notified whenever a reader event is produced. The observer MUST implement `CardReaderObserverSpi`.
Parameters	`observer` — the observer to register. MUST be non-`null`.
Throws	`IllegalArgumentException` — if the provided observer is `null`.

Signature	`removeObserver(observer: CardReaderObserverSpi) → void`
Since	1.0.0
Description	Unregisters a previously registered observer. The observer MUST stop receiving events as soon as the operation returns.
Parameters	`observer` — the observer to remove. MUST be non-`null`.
Throws	`IllegalArgumentException` — if the provided observer is `null`.

Signature	`clearObservers() → void`
Since	1.0.0
Description	Unregisters every observer currently attached to the reader.
Throws	None.

Signature	`countObservers() → integer`
Since	1.0.0
Description	Returns the number of observers currently registered with the reader.
Returns	A non-negative `integer`.
Throws	None.

Signature	`startCardDetection(detectionMode: DetectionMode) → void`
Since	1.0.0
Description	Activates card detection. Once activated, the application MAY be notified of the arrival of a card through the registered observers. The `DetectionMode` indicates the action to be followed after processing the card.
Parameters	`detectionMode` — the desired card detection mode.
Throws	`IllegalArgumentException` — if the provided detection mode is `null`.

Signature	`stopCardDetection() → void`
Since	1.0.0
Description	Stops card detection. After this operation has returned, observers SHALL NOT receive card events until detection is restarted.

Throws	None.

Signature	`finalizeCardProcessing() → void`
Since	1.0.0
Description	Notifies the observation process that the processing of the current card has been completed in order to ensure that the card monitoring cycle runs properly. This operation has no effect if the physical communication channel has already been closed. It is **mandatory** to call this operation when the physical communication channel with the card could not be closed properly. The channel closing is nominally managed during the last transmission with the card; however, there are cases where exchanges with the card are interrupted (e.g. by an exception), in which case it is necessary to explicitly close the channel using this operation. In practice, it is RECOMMENDED to invoke this operation systematically (e.g. inside a `finally` block) at the end of a card processing whatever the outcome.
Throws	None.

5.1.5.2. Enumeration DetectionMode

Nested inside `ObservableCardReader`. Card detection management options to be applied after processing a card.

Value	Since	Description
`REPEATING`	1.0.0	Continue waiting for the insertion of a next card.
`SINGLESHOT`	1.0.0	Stop and wait for a restart signal.

5.1.5.3. Enumeration NotificationMode

Nested inside `ObservableCardReader`. Options applied when a card is detected, used to decide which cards trigger observer notifications.

Value	Since	Description
`ALWAYS`	1.0.0	All cards presented to the reader are notified to the observers, regardless of the result of the selection.
`MATCHED_ONLY`	1.0.0	Only cards that have been successfully selected will be notified to the observers. The others MUST be ignored.

5.1.6. CardReaderEvent

Kind	Interface
Package	`reader`

Since	1.0.0
Purpose	Data container describing a change of state observed by a `CardReader`.

A `CardReaderEvent` carries the origin of the event (the reader name), the event type and, when available, the card selection response.

5.1.6.1. Operations

Signature	`getReaderName() → string`
Since	1.0.0
Description	Returns the name of the reader that generated the event.
Returns	A non-empty `string`.
Throws	None.

Signature	`getType() → Type`
Since	1.0.0
Description	Returns the type of the event.
Returns	A non-`null` `Type` value.
Throws	None.

Signature	`getScheduledCardSelectionsResponse() → ScheduledCardSelectionsResponse`
Since	1.0.0
Description	Returns the card selection response when it is available and `null` in all other cases. The response MAY be available when the event type is `CARD_INSERTED` and MUST be present when the event type is `CARD_MATCHED`. The returned value MUST be passed to `CardSelectionManager.parseScheduledCardSelectionsResponse` to be interpreted.
Returns	The scheduled card selections response, or `null` if the event does not carry one.
Throws	None.

5.1.6.2. Enumeration Type

Nested inside `CardReaderEvent`. Possible card reader events.

Value	Since	Description
`CARD_INSERTED`	1.0.0	A card has been inserted, with or without a specific selection.

Value	Since	Description
`CARD_MATCHED`	1.0.0	A card has been inserted that matches the selection criteria.
`CARD_REMOVED`	1.0.0	The card has been removed from the reader.
`UNAVAILABLE`	1.0.0	The reader has become unavailable.

5.1.7. ChannelControl

Kind	Enumeration
Package	`reader`
Since	2.1.0
Purpose	Policy for managing the physical channel after a card request has been executed.

Value	Since	Description
`KEEP_OPEN`	2.1.0	Leaves the physical channel open.
`CLOSE_AFTER`	2.1.0	Terminates communication with the card. The physical channel closes instantly, or a card removal sequence is initiated depending on the observation mode.

5.2. Package reader.spi

The `reader.spi` package gathers the contracts that are implemented by the **application** (as opposed to the reader implementation) when readers are observed.

5.2.1. CardReaderObserverSpi

Kind	Interface (SPI)
Package	`reader.spi`
Since	1.0.0
Purpose	Implemented by the application to receive `CardReaderEvent` instances from an `ObservableCardReader`.

5.2.1.1. Operations

Signature	`onReaderEvent(readerEvent: CardReaderEvent) → void`
Since	1.0.0
Description	Invoked when a reader event occurs. The dispatch SHOULD be performed **sequentially** and **synchronously** unless the implementation explicitly documents a different policy.
Parameters	`readerEvent` — the non-`null` event payload.

Throws	None. Implementations MUST ensure that exceptions thrown by the observer are routed to the configured exception handler.

5.2.2. CardReaderObservationExceptionHandlerSpi

Kind	Interface (SPI)
Package	`reader.spi`
Since	1.0.0
Purpose	Reader observation error handler — implemented by the application to be notified of errors occurring during the monitoring of a reader.

5.2.2.1. Operations

Signature	`onReaderObservationError(contextInfo: string, readerName: string, e: throwable) → void`
Since	1.0.0
Description	Invoked when an error occurs on the observed reader. After the operation returns, the observation process MUST be considered stopped.
Parameters	`contextInfo` — contextual information describing the operation that failed. `readerName` — name of the reader on which the error occurred. `e` — the original exception that triggered the notification.
Throws	None.

5.3. Package reader.selection

The `reader.selection` package describes the orchestration of card selection scenarios. It introduces the central `CardSelectionManager`, the family of selectors and the structures used to expose selection results.

5.3.1. CardSelector<T extends CardSelector<T>>

Kind	Interface
Package	`reader.selection`
Since	2.0.0
Purpose	Base contract for all card selectors. Defines filters used to restrict the selection process to a subset of cards.

All filters defined by this interface are **optional** and MAY be combined. If no filter is specified, any card that responds when inserted in the reader is considered selected. Conversely, if one or more filters are defined, a card SHALL NOT be selected if at least one of them rejects it.

5.3.1.1. Operations

Signature	`filterByCardProtocol(logicalProtocolName: string) → T`
Since	2.0.0
Description	Restricts the selection process to cards communicating with the reader according to a specific protocol (e.g. ISO/IEC 14443-A, ISO/IEC 14443-B, proprietary or other standardised technology). The protocol is identified by its **logical name**. **Prerequisite:** the reader MUST be a `ConfigurableCardReader` and the targeted protocol MUST have been activated through `ConfigurableCardReader.activateProtocol`.
Parameters	`logicalProtocolName` — logical name of the protocol to use as filter.
Returns	The current instance, to allow fluent composition.
Throws	`IllegalArgumentException` — if the provided logical protocol name is `null` or empty.

Signature	`filterByPowerOnData(powerOnDataRegex: string) → T`
Since	2.0.0
Description	Restricts the selection process to cards whose power-on data, as provided by the reader, matches a specific regular expression.
Parameters	`powerOnDataRegex` — the regular expression to evaluate against the power-on data.
Returns	The current instance, to allow fluent composition.
Throws	`IllegalArgumentException` — if the provided regular expression is `null`, empty or invalid.

5.3.2. BasicCardSelector

Kind	Interface
Package	`reader.selection`
Since	2.0.0
Extends	`CardSelector<BasicCardSelector>`
Purpose	Basic, technology-agnostic card selector. An instance is obtained via `ReaderApiFactory.createBasicCardSelector`.

`BasicCardSelector` does not add any operation to `CardSelector`. It exists to express the absence of ISO/IEC 7816-4-specific filters at the type level.

5.3.3. CommonIsoCardSelector<T extends CommonIsoCardSelector<T>>

Kind	Interface

Package	`reader.selection`
Since	2.0.0
Extends	`CardSelector<T>`
Purpose	Common filters used to restrict the selection process to ISO/IEC 7816-4 cards.

5.3.3.1. Operations

Signature	`filterByDfName(aid: byte array) → T`
Since	2.0.0
Description	Selects a card application DF by its name. The DF is selected only if its name starts with the provided AID, as defined by ISO/IEC 7816-4 § 4.2. The provided AID MUST be used as a parameter of the **Select Application** ISO card command.
Parameters	`aid` — the AID, encoded as a 5 to 16 byte array.
Returns	The current instance.
Throws	`IllegalArgumentException` — if the provided array is `null` or out of range.

Signature	`filterByDfName(aid: string) → T`
Since	2.0.0
Description	Selects a card application DF by its name. The AID is provided as a hexadecimal string. Semantics are otherwise identical to the byte-array overload.
Parameters	`aid` — AID encoded as a hexadecimal `string` of 5 to 16 bytes.
Returns	The current instance.
Throws	`IllegalArgumentException` — if the provided string is `null`, invalid or out of range.

Signature	`setFileOccurrence(fileOccurrence: FileOccurrence) → T`
Since	2.0.0
Description	Sets the file occurrence mode (see ISO/IEC 7816-4). The default value is `FIRST`.
Parameters	`fileOccurrence` — the navigation option to apply.
Returns	The current instance.
Throws	`IllegalArgumentException` — if `fileOccurrence` is `null`.

Signature	`setFileControlInformation(fileControlInformation: FileControlInformation) → T`
Since	2.0.0

Description	Sets the file control mode (see ISO/IEC 7816-4). The default value is `FCI`.
Parameters	`fileControlInformation` — the file control mode to apply.
Returns	The current instance.
Throws	`IllegalArgumentException` — if `fileControlInformation` is `null`.

5.3.3.2. Enumeration FileOccurrence

Nested inside `CommonIsoCardSelector`. Navigation options through the applications contained on the card according to ISO/IEC 7816-4.

Value	Since	Description
`FIRST`	2.0.0	First occurrence.
`LAST`	2.0.0	Last occurrence.
`NEXT`	2.0.0	Next occurrence.
`PREVIOUS`	2.0.0	Previous occurrence.

5.3.3.3. Enumeration FileControlInformation

Nested inside `CommonIsoCardSelector`. Types of templates returned in response to the **Select Application** command, according to ISO/IEC 7816-4.

Value	Since	Description
`FCI`	2.0.0	File Control Information.
`FCP`	2.0.0	File Control Parameters.
`FMD`	2.0.0	File Management Data.
`NO_RESPONSE`	2.0.0	No response expected.

5.3.4. IsoCardSelector

Kind	Interface
Package	`reader.selection`
Since	2.0.0
Extends	`CommonIsoCardSelector<IsoCardSelector>`
Purpose	ISO/IEC 7816-4 card selector. An instance is obtained via `ReaderApiFactory.createIsoCardSelector`.

`IsoCardSelector` does not add any operation to `CommonIsoCardSelector`.

5.3.5. CardSelectionManager

Kind	Interface

Package	`reader.selection`
Since	1.0.0
Purpose	Service responsible for preparing and executing card selection scenarios. An instance is obtained via `ReaderApiFactory.createCardSelectionManager`.

A **card selection scenario** is composed of one or more **selection cases**, each backed by a `CardSelectionExtension`. A selection case targets a specific card and MAY define additional commands to be executed after the successful selection. The behaviour of the scenario is governed by the rules below:

- If a selection case fails, the manager MUST proceed with the next selection case until none remains.
- If a selection case succeeds:
  - by default, the manager MUST stop at the first successful selection;
  - if the **multiple selection mode** is enabled (see `setMultipleSelectionMode`), the manager MUST process every remaining selection case.
- The logical channel established with the card MAY be left open (default) or closed after the selection by calling `prepareReleaseChannel`.

The manager provides three flavours of execution: explicit (synchronous) selection, scheduled (event-driven) selection, and the ability to **export/import** a selection scenario or its result for distributed architectures.

5.3.5.1. Operations

Signature	`setMultipleSelectionMode() → void`
Since	1.0.0
Description	Enables the multiple selection mode, instructing the manager to process **every** selection case even after a successful one. Disabled by default.
Throws	None.

Signature	`prepareSelection(cardSelector: CardSelector<?>, cardSelectionExtension: CardSelectionExtension) → integer`
Since	2.0.0
Description	Appends a selection case to the scenario. The returned index gives the position of the selection in the scenario (`0` for the first case, `1` for the second, etc.) and SHALL be used to retrieve the corresponding result in the `CardSelectionResult`.
Parameters	`cardSelector` — the selector containing the filters to be used. `cardSelectionExtension` — the extension used to parse the selection response.
Returns	A non-negative `integer` — the index of the appended selection case.

Throws	`IllegalArgumentException` — if either argument is `null`.

Signature	`prepareReleaseChannel() → void`
Since	1.0.0
Description	Requests the closing of the physical channel at the end of the execution of the selection scenario. This allows several selection scenarios to be chained on the same card by restarting the card connection sequence.
Throws	None.

Signature	`exportCardSelectionScenario() → string`
Since	1.1.0
Description	Exports the content of the current prepared selection scenario as a `string`. The result MAY be imported into the same or another `CardSelectionManager` via `importCardSelectionScenario`.
Returns	A non-`null` `string`.
Throws	None.

Signature	`importCardSelectionScenario(cardSelectionScenario: string) → integer`
Since	1.1.0
Description	Imports a previously exported scenario. The input MUST have been produced by `exportCardSelectionScenario`.
Parameters	`cardSelectionScenario` — the previously exported scenario.
Returns	The index of the last imported selection in the scenario.
Throws	`IllegalArgumentException` — if the string is `null` or malformed.

Signature	`processCardSelectionScenario(reader: CardReader) → CardSelectionResult`
Since	1.0.0
Description	Explicitly executes the previously prepared selection scenario against the provided reader and returns the result.
Parameters	`reader` — the reader through which the card is reached.
Returns	A non-`null` `CardSelectionResult`.

Throws	`IllegalArgumentException` — if `reader` is `null`. `ReaderCommunicationException` — if communication with the reader has failed. `CardCommunicationException` — if communication with the card has failed, or if the status word check is enabled in the card request and the card returned an unexpected code. `InvalidCardResponseException` — if the card returned invalid data during the selection process.

Signature	`scheduleCardSelectionScenario(observableCardReader: ObservableCardReader, notificationMode: ObservableCardReader.NotificationMode) → void`
Since	1.0.0
Description	Schedules the execution of the prepared selection scenario as soon as a card is presented to the supplied observable reader. Events are pushed to observers according to the requested notification mode. The result of the execution MUST be parsed via `parseScheduledCardSelectionsResponse`.
Parameters	`observableCardReader` — the reader through which the card is reached. `notificationMode` — the desired notification mode (`ALWAYS` or `MATCHED_ONLY`).
Throws	`IllegalArgumentException` — if any parameter is `null`.

Signature	`parseScheduledCardSelectionsResponse(scheduledCardSelectionsResponse: ScheduledCardSelectionsResponse) → CardSelectionResult`
Since	1.0.0
Description	Analyses the responses provided by a `CardReaderEvent` following the insertion of a card and the execution of the scheduled selection scenario.
Parameters	`scheduledCardSelectionsResponse` — the scenario execution response carried by the event.
Returns	A non-`null` `CardSelectionResult`.
Throws	`IllegalArgumentException` — if the provided response is `null`. `InvalidCardResponseException` — if the data returned by the card could not be interpreted.

Signature	`exportProcessedCardSelectionScenario() → string`
Since	1.3.0

Description	Exports the content of the previously processed selection scenario as a `string`. The result MAY be imported through `importProcessedCardSelectionScenario`. **Prerequisite:** the selection scenario MUST have been processed previously via `processCardSelectionScenario` or `parseScheduledCardSelectionsResponse`. **Caution:** if the local environment does not have all the card extensions involved in the scenario, the `CardSelectionResult` produced locally will not contain any active selection. In that case, the processed scenario MUST be exported so that it can be re-imported and interpreted on a manager which has the relevant card extensions.
Returns	A non-`null` `string`.
Throws	`IllegalStateException` — if the scenario has not yet been processed or has failed.

Signature	`importProcessedCardSelectionScenario(processedCardSelectionScenario: string) → CardSelectionResult`
Since	1.3.0
Description	Imports a previously exported **processed** selection scenario and returns the corresponding result. **Prerequisites:** the input MUST have been produced by `exportProcessedCardSelectionScenario`; the local environment MUST possess every card extension involved; the current manager MUST first have been configured with the same selection scenario as the manager that produced the export.
Parameters	`processedCardSelectionScenario` — the previously exported processed scenario.
Returns	A non-`null` `CardSelectionResult`.
Throws	`IllegalArgumentException` — if the string is `null`, malformed or contains more selection cases than the current scenario. `InvalidCardResponseException` — if the data returned by the card could not be interpreted.

5.3.6. CardSelectionResult

Kind	Interface
Package	`reader.selection`
Since	1.0.0
Purpose	Result of a selection process.

Each selection case prepared with the manager is associated with the index returned by

`prepareSelection`. The same index is used to retrieve the corresponding result here. At most one case will correspond to the **active** selected card; the operations below allow the application to exploit these results according to its needs.

5.3.6.1. Operations

Signature	`getSmartCards() → map<integer, SmartCard>`
Since	1.0.0
Description	Returns all `SmartCard` instances corresponding to successful selection cases, indexed by the value returned by `prepareSelection`.
Returns	A non-`null`, possibly empty map.
Throws	None.

Signature	`getActiveSmartCard() → SmartCard`
Since	1.0.0
Description	Returns the active matching card, i.e. the card that has been selected.
Returns	The active `SmartCard`, or `null` if there is no active card.
Throws	None.

Signature	`getActiveSelectionIndex() → integer`
Since	1.0.0
Description	Returns the index of the active selection, if any.
Returns	A non-negative `integer` if an active selection exists, `-1` otherwise.
Throws	None.

5.3.7. ScheduledCardSelectionsResponse

Kind	Interface (marker)
Package	`reader.selection`
Since	1.0.0
Purpose	Carrier for the response of the execution of a scheduled selection scenario, provided by a `CardReaderEvent`.

This interface declares no operation. It conveys the card responses to one or more of the scenario's selection cases (selection step itself plus any subsequent commands). Its content MUST be interpreted through `CardSelectionManager.parseScheduledCardSelectionsResponse`.

5.4. Package reader.selection.spi

The `reader.selection.spi` package gathers the contracts that **card extensions** implement

to plug into the selection machinery defined in Section 4.

5.4.1. CardSelectionExtension

Kind	Interface (SPI, marker)
Package	`reader.selection.spi`
Since	2.0.0
Purpose	Provided by card extensions to enrich a selection case with additional commands and to interpret the response in order to build the corresponding `SmartCard`.

This interface declares no operation at the API level; the contract between the selection manager and the extension is defined by the card extension specification consuming this SPI.

5.4.2. SmartCard

Kind	Interface (SPI)
Package	`reader.selection.spi`
Since	1.0.0
Purpose	Basic smart card whose communication has been established after a successful selection. The instance is ready to receive APDUs.

Card extensions MUST implement (and MAY extend) this interface to expose the data they have collected during the selection.

5.4.2.1. Operations

Signature	`getPowerOnData() → string`
Since	1.0.0
Description	Returns the card's power-on data, as collected by the reader at the moment the card was inserted. - For a contact reader, this is the Answer To Reset (ATR) defined by ISO/IEC 7816. - For a contactless reader, the reader decides what this data is. Some contactless readers provide a virtual ATR (partially standardised by the PC/SC standard), while other devices may have their own definition, including for example elements from the anti-collision stage of ISO/IEC 14443 (ATQA, ATQB, ATS, SAK, etc.) or any proprietary definitions. Because this data varies from one reader to another, it is exposed here as a `string` that MAY be either a hexadecimal string or any other relevant representation.
Returns	The power-on data, or `null` if no power-on data is available.
Throws	None.

5.4.3. IsoSmartCard

Kind	Interface (SPI)
Package	`reader.selection.spi`
Since	2.0.0
Extends	`SmartCard`
Purpose	ISO/IEC 7816-4 smart card whose communication has been established after a successful selection.

In addition to the inherited power-on data, an `IsoSmartCard` exposes the response to the **Select Application** command. Both the power-on data and the **Select Application** response are OPTIONAL but they MUST NOT both be absent at the same time.

5.4.3.1. Operations

Signature	`getSelectApplicationResponse() → byte array`
Since	1.0.0
Description	Returns the data received from the card in response to the **Select Application** command, including the status word.
Returns	The response bytes, or `null` if no **Select Application** command has been performed.
Throws	None.

5.5. Package reader.transaction.spi

The `reader.transaction.spi` package defines the common contract shared by all card transaction managers exposed by card extensions.

5.5.1. CardTransactionManager<T extends CardTransactionManager<T>>

Kind	Interface (SPI)
Package	`reader.transaction.spi`
Since	2.1.0
Purpose	Contains operations common to all card transactions.

To exchange data with a card, the application MUST first prepare the commands to transmit and then process them. The preparation step allows commands to be grouped to minimise network exchanges, which is especially valuable in distributed architectures. The `SmartCard` registered with the manager MUST be updated after each data exchange with the card.

5.5.1.1. Operations

Signature	`processCommands(channelControl: ChannelControl) → T`

Since	2.1.0
Description	Processes all previously prepared commands and closes the physical channel if requested. All APDUs corresponding to the prepared commands MUST be sent to the card, their responses retrieved and used to update the `SmartCard` associated with the transaction. For write commands, the `SmartCard` MUST be updated only when the command is successful. The process MUST be interrupted at the first failed command.
Parameters	`channelControl` — policy for managing the physical channel after the commands have been executed.
Returns	The current instance, to allow fluent composition.
Throws	`ReaderCommunicationException` — if a communication error with the reader occurs. `CardCommunicationException` — if a communication error with the card occurs. `InvalidCardResponseException` — if a command returns an unexpected status.

Chapter 6. Exceptions

All exception types defined by this specification are unchecked (runtime) and MAY be propagated to the application. Each exception MUST expose, at minimum, the constructors listed below.

6.1. CardCommunicationException

Kind	Runtime exception
Package	`reader`
Since	1.0.0
Purpose	Indicates that the communication with the card has failed.

6.1.1. Constructors

- `CardCommunicationException(message: string)` — *since 1.0.0*.
- `CardCommunicationException(message: string, cause: throwable)` — *since 1.0.0*.

6.2. InvalidCardResponseException

Kind	Runtime exception
Package	`reader`
Since	2.1.0
Purpose	Indicates that a response received from the card during command processing was invalid.

6.2.1. Constructors

- `InvalidCardResponseException(message: string)` — *since 2.1.0*.
- `InvalidCardResponseException(message: string, cause: throwable)` — *since 2.1.0*.

A separate `InvalidCardResponseException` MAY be defined in the `reader.selection` package for backward compatibility with selection-time errors. New code SHOULD use the type defined in the `reader` package.

6.3. ReaderCommunicationException

Kind	Runtime exception
Package	`reader`
Since	1.0.0

Purpose	Indicates that the communication with the reader has failed. The most likely cause is a physical disconnection of the reader, but other technical problems MAY also be at the origin of the failure.

6.3.1. Constructors

- `ReaderCommunicationException(message: string)` — *since 1.0.0*.
- `ReaderCommunicationException(message: string, cause: throwable)` — *since 1.0.0*.

6.4. ReaderProtocolNotSupportedException

Kind	Runtime exception
Package	`reader`
Since	1.0.0
Purpose	Indicates that the requested card protocol is not supported by the reader.

6.4.1. Constructors

- `ReaderProtocolNotSupportedException(physicalProtocolName: string)` — *since 1.0.0*. The default message MUST follow the pattern `"The protocol '<physicalProtocolName>' is not supported."`.

Chapter 7. Behavioural Requirements

This section consolidates the cross-cutting behavioural requirements that conforming implementations MUST satisfy.

7.1. Reader lifecycle

1. A `CardReader` instance MUST expose a stable name for its entire lifetime.
2. A reader instance MUST signal communication failures by raising `ReaderCommunicationException` rather than returning a default value.
3. A reader instance MUST refuse to be observed until a `CardReaderObservationExceptionHandlerSpi` has been registered.

7.2. Observation lifecycle

1. Observer notifications MUST be delivered **sequentially** per reader; concurrent invocations of `onReaderEvent` on the same reader MUST NOT occur.
2. After `stopCardDetection`, observers SHALL NOT receive new events until `startCardDetection` has been called again.
3. Any uncaught exception raised inside an observer MUST be relayed to the observation exception handler.

7.3. Selection scenario lifecycle

1. A scenario is considered "prepared" once at least one selection case has been appended via `prepareSelection`.
2. A scenario MAY be executed multiple times, but each execution operates on the scenario as it stood at the time of the call.
3. `exportProcessedCardSelectionScenario` MUST raise `IllegalStateException` if no scenario has been processed yet.

7.4. Channel control

1. The default channel policy after a successful selection is `KEEP_OPEN`.
2. After `prepareReleaseChannel` has been invoked, the next execution of the scenario MUST close the physical channel at the end of the selection.
3. `finalizeCardProcessing` MUST be idempotent.

Appendix A: Change Log — 3.0.0-SNAPSHOT (Working Draft)

This appendix tracks, on a best-effort basis, the deltas introduced by the 3.0.0-SNAPSHOT working draft compared to the previous published release. The list below MUST be kept in sync with `api_class_diagram_diff.puml`.

- *Pending: list of additions, deprecations and removals as agreed by the editorial group.*

Appendix B: Traceability Matrix (Reference Java Binding)

The table below maps each abstract element defined in this specification to the corresponding identifier in the reference Java binding hosted at `eclipse-keypop/keypop-reader-java-api`.

Section	Element	Java type / member	Notes
Section 5.1.2	`ReaderApiFactory`	`org.eclipse.keypop.reader.ReaderApiFactory`	—
Section 5.1.1	`ReaderApiProperties.VERSION`	`org.eclipse.keypop.reader.ReaderApiProperties.VERSION`	Reference value: `"2.1"` in the 2.1.x binding.
Section 5.1.3	`CardReader`	`org.eclipse.keypop.reader.CardReader`	—
Section 5.1.4	`ConfigurableCardReader`	`org.eclipse.keypop.reader.ConfigurableCardReader`	—
Section 5.1.5	`ObservableCardReader`	`org.eclipse.keypop.reader.ObservableCardReader`	Includes nested `DetectionMode`, `NotificationMode`.
Section 5.1.6	`CardReaderEvent`	`org.eclipse.keypop.reader.CardReaderEvent`	Includes nested `Type`.
Section 5.1.7	`ChannelControl`	`org.eclipse.keypop.reader.ChannelControl`	—
Section 5.2.1	`CardReaderObserverSpi`	`org.eclipse.keypop.reader.spi.CardReaderObserverSpi`	—
Section 5.2.2	`CardReaderObservationExceptionHandlerSpi`	`org.eclipse.keypop.reader.spi.CardReaderObservationExceptionHandlerSpi`	—
Section 5.3.1	`CardSelector<T>`	`org.eclipse.keypop.reader.selection.CardSelector`	—
Section 5.3.2	`BasicCardSelector`	`org.eclipse.keypop.reader.selection.BasicCardSelector`	—
Section 5.3.3	`CommonIsoCardSelector<T>`	`org.eclipse.keypop.reader.selection.CommonIsoCardSelector`	—

Section	Element	Java type / member	Notes
Section 5.3.4	`IsoCardSelector`	`org.eclipse.keypop.reader.selection.IsoCardSelector`	—
Section 5.3.5	`CardSelectionManager`	`org.eclipse.keypop.reader.selection.CardSelectionManager`	—
Section 5.3.6	`CardSelectionResult`	`org.eclipse.keypop.reader.selection.CardSelectionResult`	—
Section 5.3.7	`ScheduledCardSelectionsResponse`	`org.eclipse.keypop.reader.selection.ScheduledCardSelectionsResponse`	Marker interface.
Section 5.4.1	`CardSelectionExtension`	`org.eclipse.keypop.reader.selection.spi.CardSelectionExtension`	Marker SPI.
Section 5.4.2	`SmartCard`	`org.eclipse.keypop.reader.selection.spi.SmartCard`	—
Section 5.4.3	`IsoSmartCard`	`org.eclipse.keypop.reader.selection.spi.IsoSmartCard`	—
Section 5.5.1	`CardTransactionManager<T>`	`org.eclipse.keypop.reader.transaction.spi.CardTransactionManager`	—
Section 6.1	`CardCommunicationException`	`org.eclipse.keypop.reader.CardCommunicationException`	—
Section 6.2	`InvalidCardResponseException`	`org.eclipse.keypop.reader.InvalidCardResponseException`	—
Section 6.3	`ReaderCommunicationException`	`org.eclipse.keypop.reader.ReaderCommunicationException`	—
Section 6.4	`ReaderProtocolNotSupportedException`	`org.eclipse.keypop.reader.ReaderProtocolNotSupportedException`	—

Appendix C: Copyright and License

The reference Java binding of this API is itself distributed under the MIT License and is part of the Eclipse Keypop project hosted by the Eclipse Foundation.